

AFRILANG Stdlib

std/c/parsebinary

Bibliothèque AFRILANG `std/c/parsebinary` (niveau complex, catégorie complex). Elle expose 7 fonction(s) :
binaLen, bina

```
`import "std/c/parsebinary" . `use parsebinary`
```

- `binaLen(s text) ? number`
- `binaUpper(s text) ? text`
- `binaLower(s text) ? text`
- `binaTrim(s text) ? text`
- `binaSplit(s text, delim text) ? list text`
- `binaJoin(parts list text, delim text) ? text`
- `binaReplace(s text, from text, to text) ? text`

Category: complex | Tier: complex | Functions: 7

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/parsebinary` (niveau complexe, catégorie complexe). Elle expose 7 fonction(s) : binaLen, bina

```
`import "std/c/parsebinary"` . `use parsebinary`  
- `binaLen(s text) ? number`  
- `binaUpper(s text) ? text`  
- `binaLower(s text) ? text`  
- `binaTrim(s text) ? text`  
- `binaSplit(s text, delim text) ? list text`  
- `binaJoin(parts list text, delim text) ? text`  
- `binaReplace(s text, from text, to text) ? text`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/parsebinary"  
use parsebinary
```

Niveau : complexe · Catégorie : Modules complexe (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? binaLen(s text) ? number  
? binaUpper(s text) ? text  
? binaLower(s text) ? text  
? binaTrim(s text) ? text  
? binaSplit(s text, delim text) ? list  
? binaJoin(parts list text, delim text) ? text  
? binaReplace(s text, from text, to text) ? text
```

4. Exemples d'utilisation

```
import "std/c/parsebinary"  
use parsebinary  
  
create result = binaLen(/* args */)   
say result  
  
create result = binaUpper(/* args */)   
say result  
  
create result = binaLower(/* args */)   
say result  
  
create result = binaTrim(/* args */)   
say result  
  
create result = binaSplit(/* args */)   
say result  
  
create result = binaJoin(/* args */)   
say result
```

5. Bonnes pratiques

? Préférez `import "std/c/parsebinary"` en tête de fichier.
? Lancez `afirilang check` avant de livrer.
? Combinez avec les modules stdlib de la même catégorie.
? Voir /stdlib/ pour le source live et les démos playground.
? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

module parsebinary

```
function binaLen(s text) returns number
end

function binaUpper(s text) returns text
end

function binaLower(s text) returns text
end

function binaTrim(s text) returns text
end

function binaSplit(s text, delim text) returns list text
end

function binaJoin(parts list text, delim text) returns text
end

function binaReplace(s text, from text, to text) returns text
end
end
```

ENGLISH

1. Overview

AFRILANG library ``std/c/parsebinary`` (tier complex, category complex). Exports 7 function(s): `binaLen`, `binaUpper`, `binaLo`

```
`import "std/c/parsebinary"` . `use parsebinary`  
- `binaLen(s text) ? number`  
- `binaUpper(s text) ? text`  
- `binaLower(s text) ? text`  
- `binaTrim(s text) ? text`  
- `binaSplit(s text, delim text) ? list text`  
- `binaJoin(parts list text, delim text) ? text`  
- `binaReplace(s text, from text, to text) ? text`
```

2. Installation & import

Add the module to your program:

```
import "std/c/parsebinary"  
use parsebinary
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? binaLen(s text) ? number  
? binaUpper(s text) ? text  
? binaLower(s text) ? text  
? binaTrim(s text) ? text  
? binaSplit(s text, delim text) ? list  
? binaJoin(parts list text, delim text) ? text  
? binaReplace(s text, from text, to text) ? text
```

4. Usage examples

```
import "std/c/parsebinary"  
use parsebinary  
  
create result = binaLen(/* args */)   
say result  
  
create result = binaUpper(/* args */)   
say result  
  
create result = binaLower(/* args */)   
say result  
  
create result = binaTrim(/* args */)   
say result  
  
create result = binaSplit(/* args */)   
say result  
  
create result = binaJoin(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/parsebinary"` at the top of your file.  
? Run `afriLang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module parsebinary
```

```
function binaLen(s text) returns number
end

function binaUpper(s text) returns text
end

function binaLower(s text) returns text
end

function binaTrim(s text) returns text
end

function binaSplit(s text, delim text) returns list text
end

function binaJoin(parts list text, delim text) returns text
end

function binaReplace(s text, from text, to text) returns text
end
end
```